

SwingBy — Master Context Prompt for Claude Code

What is SwingBy

SwingBy is a dual-sided service marketplace app connecting service providers ("Businesses") with people seeking services ("Clients"). Think Uber meets Thumbtack meets Facebook Marketplace — built for Calgary, expanding to North America.

Core Model

Two user roles — one app

- **Business** = service provider (solo side-hustler or company with employees)
- **Client** = person needing a service

Two discovery flows

1. **Geo-browse** — client opens map, sees nearby businesses available now
2. **Post & match** — client posts a request, businesses express interest, client accepts or rejects

Booking flow

Client posts → Businesses click "Interested" → Client accepts/rejects → Booked → Employee assigned → Confirmed

Safety layer

When booking confirms, client sees the exact person showing up — name, photo, role. Even if the business has 5 employees, the assigned worker's profile is pushed to the client. (Uber model)

Payment — Escrow

- Client pays 100% upfront
- 50% released to Business immediately on confirmation
- 50% held in escrow until job marked complete
- SwingBy takes commission cut (10%) before payout

Anti-spam

Client is always in control. Businesses can only express interest — client accepts or rejects. No direct contact before confirmation.

Business accounts

- Owner account = 1 login
- Owner creates child accounts for employees
- Owner manually assigns employees to jobs

Service posts

- Expire after 7 days
- Client sets budget
- Business can accept budget or counter-quote

In-app chat

Only unlocked after booking is confirmed.

Reviews

Two-way — client reviews business, business reviews client. Tied to completed bookings only.

Cancellations

Penalty applies. Amount based on proximity to confirmed date.

Tech Stack

Layer	Tech
Backend	FastAPI (Python)
Database	PostgreSQL via Supabase
Mobile	React Native + Expo
Web Dashboard	React + Vite
Auth	Supabase Auth
Storage	AWS S3
Maps	Google Maps API
Push Notifications	Expo Push + FCM

Layer	Tech
Redis	Post-MVP only

Monorepo Structure

```
SwingBy/
├── .github/workflows/
├── backend/                               ← FastAPI (Python)
│   ├── app/
│   │   ├── api/
│   │   │   ├── __init__.py
│   │   │   ├── auth.py                 ← DONE
│   │   │   ├── businesses.py          ← IN PROGRESS
│   │   │   ├── models/
│   │   │   ├── schemas/
│   │   │   ├── services/
│   │   │   ├── database.py             ← DONE
│   │   │   └── main.py                 ← DONE
│   ├── requirements.txt
│   └── .env                             ← NOT committed to git
├── mobile/                               ← React Native + Expo
│   ├── src/
│   │   ├── screens/
│   │   ├── components/
│   │   ├── navigation/
│   │   └── services/
│   ├── app.json
│   └── package.json
├── web/                                  ← React + Vite dashboard
│   ├── src/
│   │   ├── pages/
│   │   ├── components/
│   │   └── services/
│   ├── index.html
│   └── package.json
├── docs/
│   └── swingby_database_schema.md
├── .gitignore                            ← includes CODE/ and research/
└── README.md
```

Database — 10 Tables in Supabase (PostgreSQL)

All tables have RLS enabled. UUID primary keys. Created and live in Supabase.

USERS

sql

```
create table users (  
  id uuid primary key default gen_random_uuid(),  
  first_name text not null,  
  last_name text not null,  
  email text unique not null,  
  phone text,  
  role text not null check (role in ('client', 'business_owner', 'employee')),  
  avatar_url text,  
  created_at timestamptz default now()  
);
```

BUSINESSES

sql

```
create table businesses (  
  id uuid primary key default gen_random_uuid(),  
  owner_id uuid not null references users(id) on delete cascade,  
  business_name text not null,  
  category text not null,  
  custom_category text,  
  description text,  
  license_number text,  
  license_status text default 'pending' check (license_status in ('pending', 'verifi  
  lat float,  
  lng float,  
  service_radius_km float default 25,  
  avg_rating float default 0,  
  review_count int default 0,  
  created_at timestamptz default now()  
);
```

EMPLOYEES

sql

```
create table employees (  
  id uuid primary key default gen_random_uuid(),  
  business_id uuid not null references businesses(id) on delete cascade,  
  user_id uuid not null references users(id) on delete cascade,  
  role_title text,  
  avatar_url text,  
  is_active boolean default true,  
  created_at timestamptz default now()  
);
```

SERVICE_POSTS

sql

```
create table service_posts (  
  id uuid primary key default gen_random_uuid(),  
  client_id uuid not null references users(id) on delete cascade,  
  title text not null,  
  description text,  
  category text not null,  
  budget float not null,  
  status text default 'open' check (status in ('open', 'matched', 'expired', 'cancel  
  lat float,  
  lng float,  
  expires_at timestamptz default now() + interval '7 days',  
  created_at timestamptz default now()  
);
```



INTERESTS

sql

```
create table interests (  
  id uuid primary key default gen_random_uuid(),  
  post_id uuid not null references service_posts(id) on delete cascade,  
  business_id uuid not null references businesses(id) on delete cascade,  
  quoted_price float,  
  status text default 'pending' check (status in ('pending', 'accepted', 'rejected'))  
  created_at timestamptz default now()  
);
```

BOOKINGS

sql

```
create table bookings (  
  id uuid primary key default gen_random_uuid(),  
  client_id uuid not null references users(id) on delete cascade,  
  business_id uuid not null references businesses(id) on delete cascade,  
  employee_id uuid references employees(id) on delete set null,  
  post_id uuid references service_posts(id) on delete set null,  
  service_category text not null,  
  total_amount float not null,  
  commission_rate float default 0.10,  
  platform_fee float,  
  status text default 'confirmed' check (status in ('confirmed', 'in_progress', 'com  
  payment_status text default 'held' check (payment_status in ('held', 'partial_rele  
  proposed_date_1 timestamptz,  
  proposed_date_2 timestamptz,  
  proposed_date_3 timestamptz,  
  confirmed_date timestamptz,  
  created_at timestamptz default now()  
);
```

MESSAGES

sql

```
create table messages (  
  id uuid primary key default gen_random_uuid(),  
  booking_id uuid not null references bookings(id) on delete cascade,  
  sender_id uuid not null references users(id) on delete cascade,  
  content text not null,  
  sent_at timestamptz default now()  
);
```

PAYMENTS

sql

```
create table payments (  
  id uuid primary key default gen_random_uuid(),  
  booking_id uuid not null references bookings(id) on delete cascade,  
  total_charged float not null,  
  escrow_held float not null,  
  released_to_business float default 0,  
  platform_cut float not null,  
  stripe_payment_intent_id text,  
  status text default 'held' check (status in ('processing', 'held', 'partial', 'com  
  released_at timestamptz,  
  created_at timestamptz default now()  
);
```

REVIEWS

sql

```
create table reviews (  
  id uuid primary key default gen_random_uuid(),  
  booking_id uuid not null references bookings(id) on delete cascade,  
  reviewer_id uuid not null references users(id) on delete cascade,  
  reviewee_id uuid not null,  
  reviewee_type text not null check (reviewee_type in ('client', 'business')),  
  rating int not null check (rating >= 1 and rating <= 5),  
  comment text,  
  created_at timestamptz default now()  
);
```

CANCELLATIONS

```
sql

create table cancellations (
  id uuid primary key default gen_random_uuid(),
  booking_id uuid not null references bookings(id) on delete cascade,
  cancelled_by uuid not null references users(id) on delete cascade,
  reason text,
  penalty_amount float default 0,
  created_at timestamptz default now()
);
```

Backend — What's Built

backend/app/database.py — DONE

Connects FastAPI to Supabase via SQLAlchemy. Reads DATABASE_URL from .env.

backend/app/main.py — DONE

FastAPI app with CORS middleware. Health check route confirms database connection. Auth and businesses routers registered.

backend/app/api/auth.py — DONE

- `POST /auth/signup` — creates Supabase auth user + inserts into users table
- `POST /auth/login` — returns access token

backend/app/api/businesses.py — IN PROGRESS

- `POST /businesses/create`
 - `GET /businesses/`
 - `GET /businesses/{business_id}`
 - `PATCH /businesses/{business_id}`
-

What Needs to Be Built Next

API endpoints still needed:

1. `backend/app/api/employees.py` — create, list, deactivate employees

2. `backend/app/api/service_posts.py` — create post, list open posts, expire posts
3. `backend/app/api/interests.py` — express interest, accept, reject
4. `backend/app/api/bookings.py` — create booking, assign employee, mark complete
5. `backend/app/api/payments.py` — escrow logic, release payment
6. `backend/app/api/reviews.py` — create review, list reviews
7. `backend/app/api/messages.py` — send message, list messages

Mobile app (React Native + Expo) — not started

Web dashboard (React + Vite) — not started

Environment Variables (.env — never commit this)

```
DATABASE_URL=postgresql://postgres.PROJECT_ID:PASSWORD@HOST:5432/postgres
SECRET_KEY=your_secret_key
SUPABASE_URL=https://PROJECT_ID.supabase.co
SUPABASE_KEY=sb_publishable_...
AWS_BUCKET=
```

How to Run the Backend

```
bash
cd backend
"C:/Python314/python.exe" -m uvicorn app.main:app --reload
```

API docs available at: `http://127.0.0.1:8000/docs`

Key Design Decisions

- One USERS table, three roles — simpler auth, role checked on every protected route
- EMPLOYEES are child accounts — owner creates them, assigns them to jobs manually
- INTERESTS as spam shield — businesses express interest, client accepts or rejects
- `post_id` is nullable on BOOKINGS — supports both flows: post-and-match AND direct browse

- Escrow split in PAYMENTS — 50% on confirmation, 50% on completion, platform fee taken before payout
 - MESSAGES locked to confirmed BOOKINGS only
 - LICENSE_STATUS is manual — starts as pending, SwingBy team verifies manually, auto-verification post-MVP
 - expires_at on SERVICE_POSTS — 7 day limit
-

GitHub

Repo: <https://github.com/4alkubati/SWINGBY> Branch: main